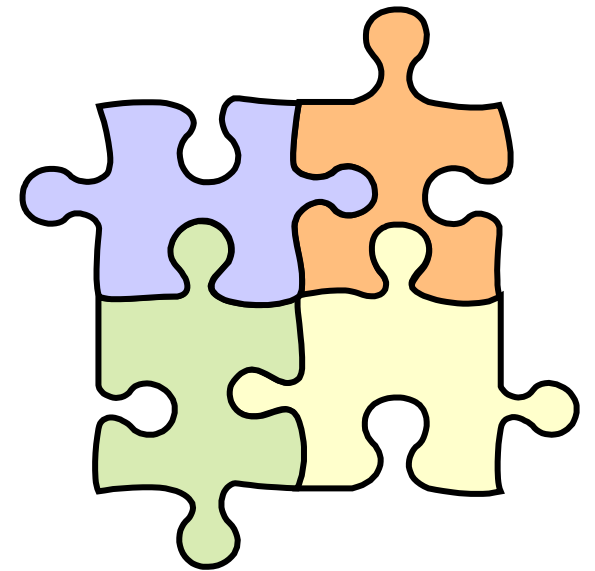
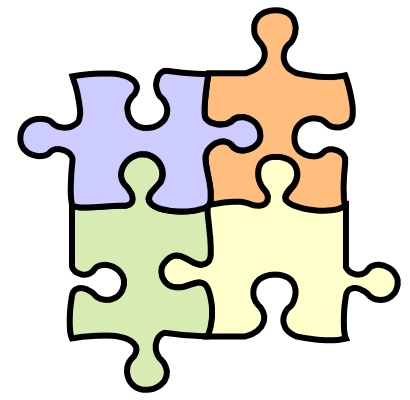


- Lab Info
 - [Lab 1](#)
 - [Lab 2](#)
 - [Lab 3](#)

[Link to Lecture Notes](#)





Lab 1



Lab 1 :: Prep == Play!

humble

```
sudo apt install ros-$ROS_DISTRO-turtlebot3-gazebo
```

```
export TURTLEBOT3_MODEL=burger
```

```
ros2 launch turtlebot3_gazebo empty_world.launch.py
```

Replace 'humble' with your actual ROS 2 version if different

```
sudo apt update
```

```
sudo apt install ros-humble-turtlebot3
```

```
export TURTLEBOT3_MODEL=burger
```

```
ros2 run turtlebot3_teleop teleop_keyboard
```

```
source /opt/ros/humble/setup.bash
```

```
ros-admin@ros-setup:~$ ros2 launch turtlebot3_gazebo
```

`empty_world.launch.py`

`turtlebot3_world.launch.py`

`turtlebot3_house.launch.py`

turtlesim ≠ turtlebot3 (sim & real)

Terminal 2: Start Teleop (The Controller)

Run the keyboard control node so you can move the robot around. Keep this window focused to use your keys.

bash

```
export TURTLEBOT3_MODEL=burger
```

```
ros2 run turtlebot3_teleop teleop_keyboard
```

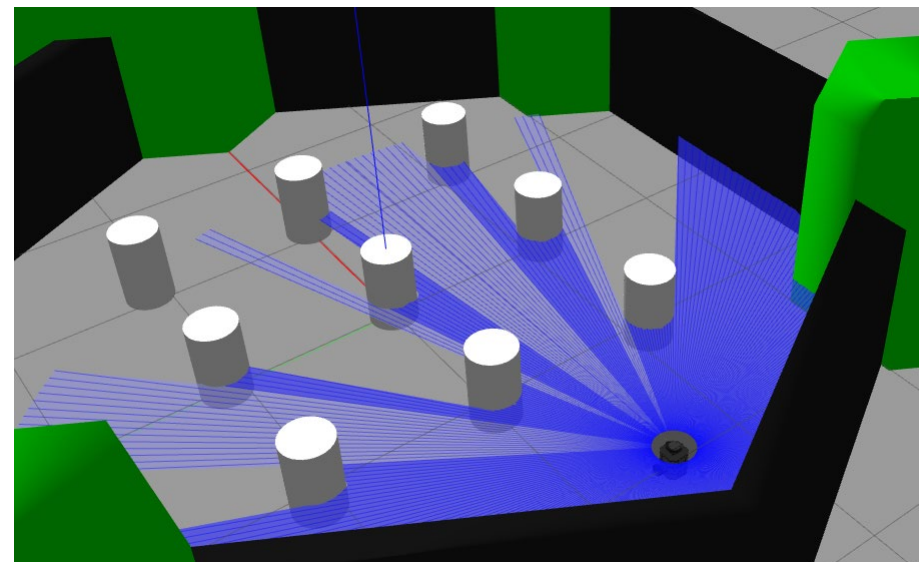
Terminal 3: Start RViz2 (The Visualization)

Launch the visualizer to see the sensor data live while you move.

bash

```
export TURTLEBOT3_MODEL=burger
```

```
ros2 launch turtlebot3_bringup rviz2.launch.py
```



There are 4 Turtlebots {red, blue, green, white} with a remote laptop each ... sit down and remember your <Color> ...

- Insert a battery into your turtlebot; switch on the turtlebot; place the turtlebot on a „stand“ on the ground
- Remote laptop login: user **eit**, password **electron**
- Open a terminal and ping your turtlebot, adjust the command to your <Color> :

```
eit@BlueDevelop:~$ ping bluebot
```

If you receive an answer, all is fine; kill the ping (CTRL-C) ... otherwise call for help.
- Login to the turtlebot via ssh:

```
eit@BlueDevelop:~$ ssh -l obv bluebot
```

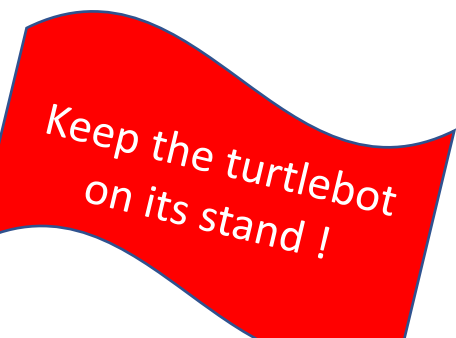
User **obv**, password **turtle01**; if the login works, all is fine ... otherwise call for help. Keep the shell open.
In this shell you are working on the Single Board Computer (SBC, Raspberry Pi) on the turtlebot.
Launch the boot script on the turtlebot:

```
obv@bluebot:~$ ros2 launch turtlebot3_bringup robot.launch.py
```
- Open a second shell. Check out the nodes, topics, services:

```
eit@BlueDevelop:~$ ros2 node list
```

...
- **Write** down (photograph) the number of nodes, topics, services.

- Start rqt. Move the window to the big screen.
`eit@BlueDevelop:~$ rqt`
- Open the Message Publisher plugin. Choose the topic `/cmd_vel`. Set the parameters and publish the message (check box).
`linear.x=0.1` or `angular.z=0.4` Debug until the robot moves.
- Open a third shell. Observe the published messages via `eit@BlueDevelop:~$ ros2 topic echo /cmd_vel`
- Stop the motion by publishing zeros.
- Use values for `linear.x` and `angular.z` so that the robot moves „medium speed“. Write down the values.
- Open the Service Caller plugin. Choose the Service `/sound` and call the service. Write down what happens.
- Go to an available shell on BlueDevelop. Find out about the request and response arguments of the `/sound` service. Write them down.



What are
„reasonable“
values for the
/cmd_vel?

For a **TurtleBot3 Burger**, reasonable values for the `/cmd_vel` topic are strictly bound by the physical limits of its [DYNAMIXEL XL430-W250 actuators](#).

The `/cmd_vel` topic expects a message type of `geometry_msgs/Twist`. Because the TurtleBot3 Burger is a **differential-drive robot**, it can only move forward/backward (x -axis) and rotate around its center (z -axis). All other fields (y and z for linear, x and y for angular) must remain 0.0.

Target Boundaries for Twist Fields

Velocity Component	Field Name	Minimum Value	Maximum Value	Unit	Description
Linear Velocity	<code>linear.x</code>	-0.22	+0.22	Meters per second (<i>m/s</i>)	Forward (+) and Backward (-)
Angular Velocity	<code>angular.z</code>	-2.84	+2.84	Radians per second (<i>rad/s</i>)	Counter-Clockwise (+) and Clockwise (-)

read
me
later

What are
„reasonable“
values for the
/cmd_vel?

not
true

Important Practical Insights

- **The Hardware Clamp:** If you publish values outside this range (e.g., `linear.x = 1.0`), the onboard [TurtleBot3 core locomotion node](#) will safely **clamp the output** to the maximum limits (± 0.22 or ± 2.84). However, setting them excessively high can cause unintended path calculations in higher-level software stacks.
- **Effective Max Truncation:** Due to data type casting inside the [OpenCR microcontroller firmware](#), the highest achievable effective linear velocity tops out around **0.21 m/s**, and angular velocity tops out around **2.63 rad/s**.
- **Safe Starting Default:** If you are testing custom code or scripts via a publisher, a highly reasonable "slow and steady" testing value is `linear.x = 0.08` and `angular.z = 0.5`.

read
me
later

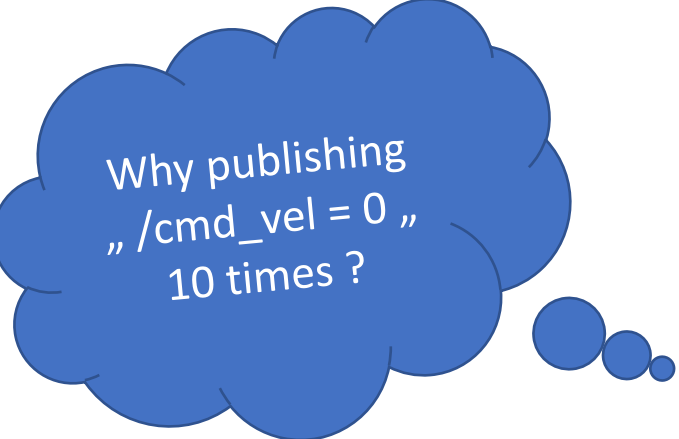
See the
lecture slides
for help

- Create a package `lab1pkg`.
- Create the Python file `stop_turtle` with a class `StopTurtle` that creates the node `stop_turtle_node`.
 - The node should publish all zeros to the `/cmd_vel` topic.
 - Execute the `publish(cmd)` multiple times with **0.2 secs pause** in between: `for n in range(1,10) ... sleep(0.2)`
 - Develop small increments.
 - Start with a simple logger output that the node was started.
 - To make sure things work: publish a value larger than zero, and only eventually zero.
- Define the executable `stop_turtlebot3` in `setup.py`.
- Test the node by moving the turtlebot forward (via `rqt`) and stopping it via the new node.

Naming:

Package	<code>lab1pkg</code>
Source *.py	<code>stop_turtle</code>
Class	<code>StopTurtle</code>
Executable	<code>stop_turtlebot3</code>
Node	<code>stop_turtle_node</code>

Keep the turtlebot
on its stand !



Why publishing
„ /cmd_vel = 0 „
10 times ?

This is a very common issue with the TurtleBot3 Burger. It happens because of how the **OpenCR microcontroller firmware handles command timeouts and message queuing**, combined with Wi-Fi network latency.

The core reasons behind this behavior outline why your workaround is necessary, followed by a way to permanently fix it in code.

1. The Underlying Causes

- **The Command Timeout (Watchdog Mechanism):** The OpenCR board running the hardware loop on the Burger has a built-in safety timeout (usually set to `0.5` or `1.0` seconds). If it does not receive a continuous stream of `/cmd_vel` messages, it assumes communication was lost and maintains the last known command until that timeout expires, or it stops automatically.
- **Packet Loss & Network Latency:** Because the TurtleBot3 communicates over Wi-Fi, sending a **single** `/cmd_vel` zero message is highly risky. If that exact Wi-Fi packet is dropped or delayed, the robot keeps driving using the last velocity stored in its buffer.
- **The "Latching" Effect of Single Publications:** If you publish a message only once (e.g., using a terminal command or a single script call without a loop), the TurtleBot's onboard processing node may fail to register it instantly if it is busy processing LiDAR arrays or odometry calculations.

By sending it 10 times in a loop, you guarantee that at least one of those packets bypasses network noise and hits the OpenCR message queue.



read
me
later



Why publishing
„ /cmd_vel = 0 „
10 times ?

2. The Proper ROS 2 Solution (The Constant Loop)

In ROS 2, you should **never** publish a single, one-off `/cmd_vel` message. The standard practice is to run a continuous publishing timer loop (usually at **10 Hz** or **20 Hz**). 

Instead of changing your variables and sleeping, your node should constantly broadcast the current target velocity—even if that velocity is zero.

3. Quick Terminal Test

If you want to stop the robot manually via the terminal, avoid using a single `ros2 topic pub` command. Instead, force it to stream at a constant rate using the `-r` flag:



```
ros2 topic pub -r 20 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

The `-r 20` tells ROS 2 to send the zero command 20 times per second, which will stop the Burger instantly without failing.

- Create another node in the package lab1pkg.
 - The node should move the turtlebot forward/backward and turn left/right using „medium speed“ random values.
 - Change the values every 2 seconds (Timer with Callback Function)
 - Make sure to include the dependencies in package.xml (geometry_msgs, ...) and to update setup.py.
- Test the node while keeping the turtlebot on its stand. Can you stop the bot with your stop_turtlebot3 program?
- If you feel that everything works fine and you can somehow stop the bot before any collision, take the turtlebot off the stand and have it drive around in the lab ...
- Put the turtlebot back on its „stand“.

Naming:

Package	lab1pkg
Source *.py	randwalk_turtle
Class	RandwalkTurtle
Executable	randwalk_turtlebot3
Node	randwalk_turtle_node

- By using the „means of your choice“, learn about the laser scanner. You might
 - review code obtained from the „world brain“ (Internet, AI)
 - play with **rqt**
 - visualize scanner data via **rviz2**
 - think about a subscriber to laser scanner data
 - experiment with „blocking the view“ of the sensor in various distances
- Do not look straight into the sensor (although it is eye safe).
- Do not spend too much time on this task 😊

Use the appropriate QoS profile:

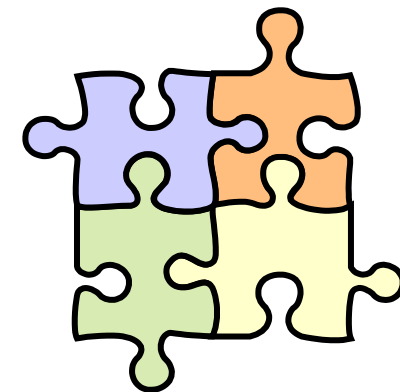
```
qos = rclpy.qos.QoSProfile(...BEST_EFFORT, ... KEEP_LAST..., depth=1)  
create_subscription(LaserScan, "/scan ", self.scan_callback, qos)
```

- Create another node in the package lab1pkg.
 - The node should move the turtlebot forward/backward and turn as done in RandwalkTurtle(), but now use the laser scanner data to prevent collisions.
 - Think about a reasonable control strategy in case another object is too close to the turtlebot.
- Test the node while keeping the turtlebot on its stand. Use your foot or something else to check out the collision detection and avoidance strategy.
- If you feel that everything works fine, take the turtlebot off the stand and have it drive around in the lab ...
- If you are bored and you do not want to go home: Call the /sound service whenever another object is too close.
- Put the turtlebot back on its „stand“.

Naming:

Package	lab1pkg
Source *.py	randwalk_safe_turtle
Class	RandwalkSafeTurtle
Executable	randwalk_safe_turtlebot3
Node	randwalk_safe_turtle_node

- Upload the source files that you completed to Moodle (one team member is sufficient)
 - `stop_turtle.py`
 - `randwalk_turtle.py`
 - `randwalk_safe_turtle.py`
- No worries if you did not finish all three ... we continue in Lab2
- There is no Lab1 Report !



Lab 2

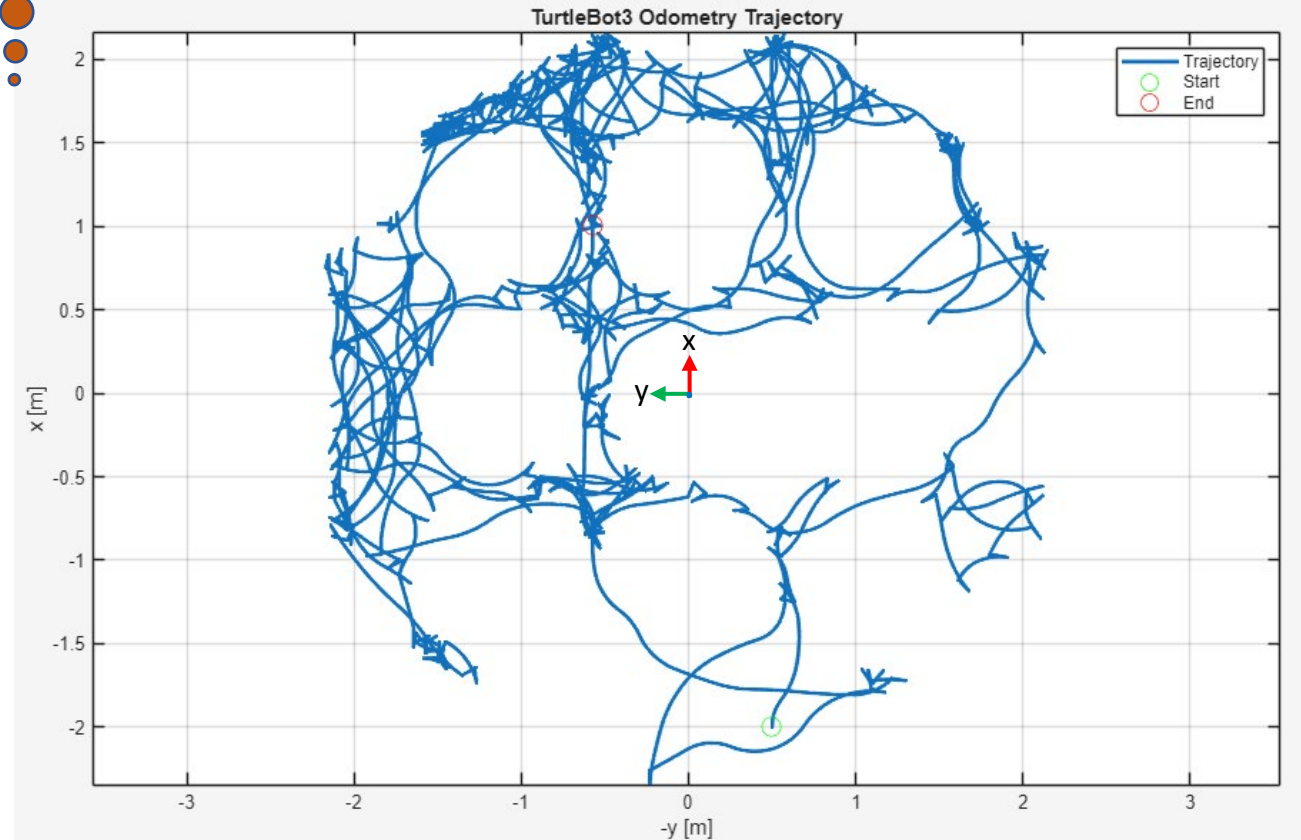
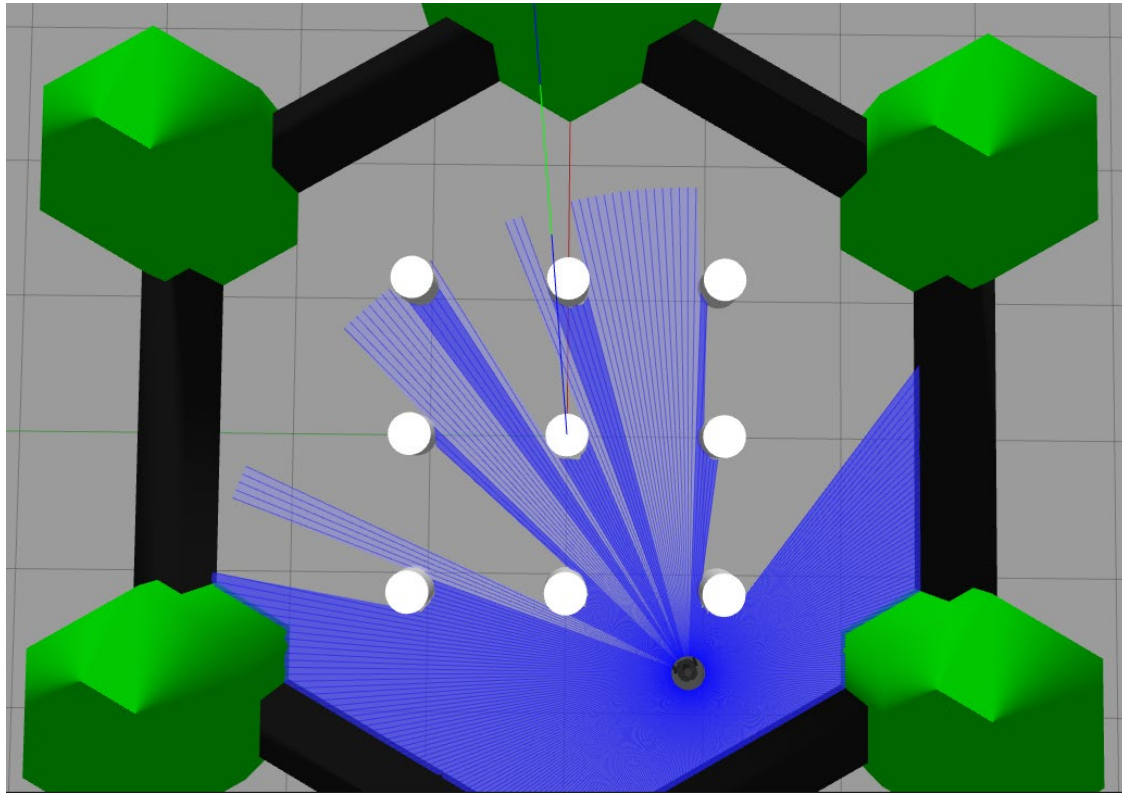


- Read the slides on the laser scanner in the lecture notes
 - Section „ROS :: Laserscan Data“
- Finish the program „RandwalkSafeTurtle“ from Lab1
- Test the program in the Gazebo simulation environment „turtlebot3_world“
- Bring the finished and well tested program to Lab2 ...

- The Lab2 Report is a zip file to be uploaded by each student to Moodle
(the lab teams can upload the same report)
- The zip file contains
 - `randwalk_safe_turtle.py`
 - a pdf file answering the questions from the next slide
- Deadline for the upload to Moodle: 30-Jun-2026, 23:59

- When you publish a message or call a service, why does only your turtlebot (Color) respond and not any other?
- Is using the laser scanner a reliable strategy to avoid collisions?
- What is the detection range of the laser scanner?
- How is the index of the laser scan data mapped to the turtlebot coordinates?
- What are the other components of the /cmd_vel message good for?
- Explain BEST_EFFORT and KEEP_LAST in the context of QoS.
- Visualize the timing of your program (rough sketch).
This should explain when new sensor data is processed, when the bot stops, and when it gets a new /cmd_vel.
- What are the weaknesses of your `randwalk_safe_turtle.py` solution ?

what is
ground truth ?

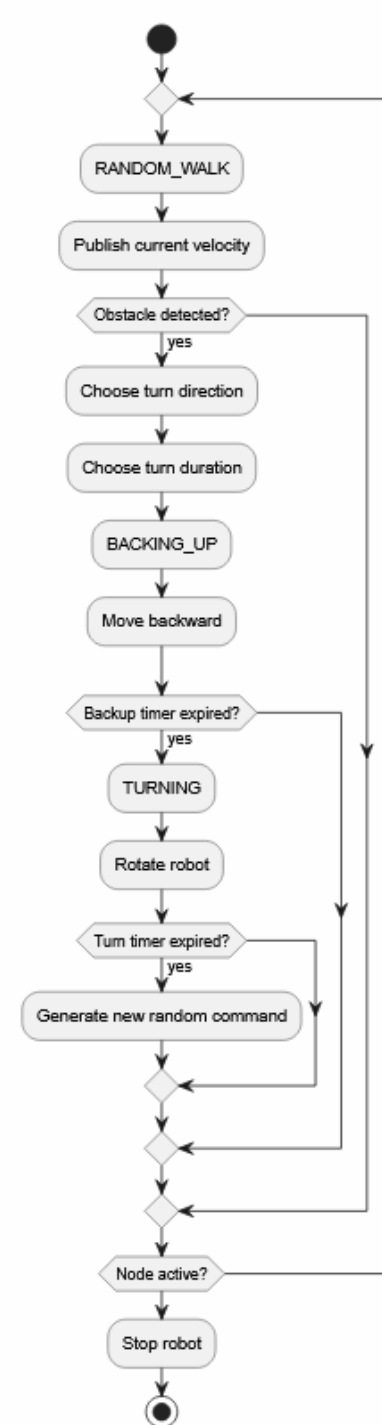
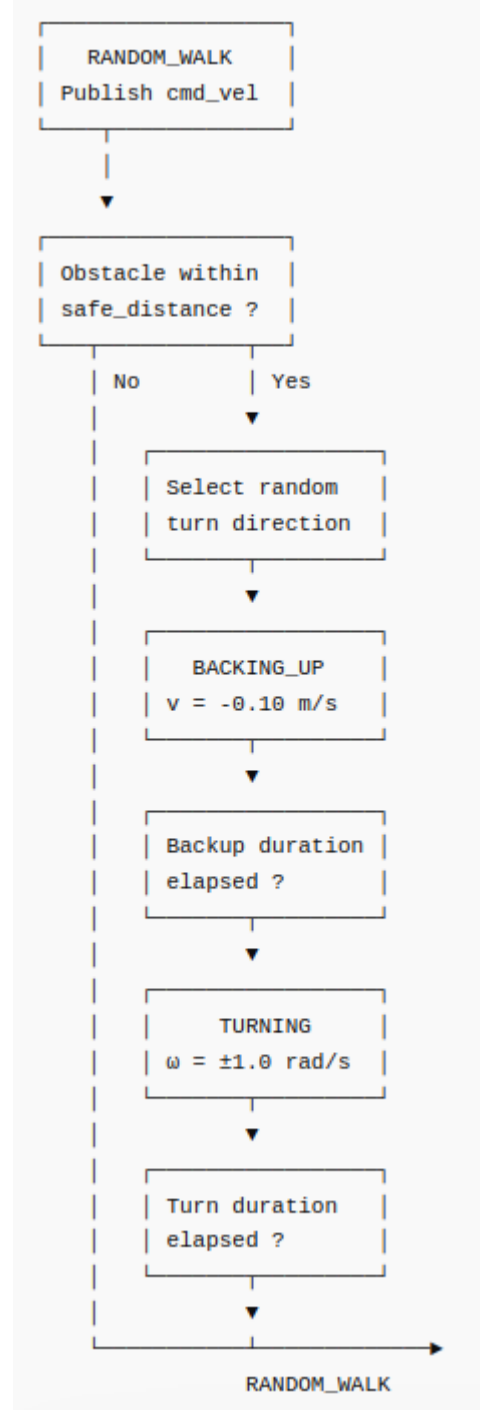


```
$ ros2 bag record -o randWalk7 /odom
```

extract data from bag and write to csv file:

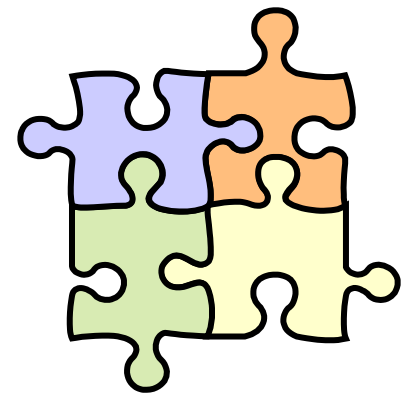
```
extract_odom.py
```

plot csv data in Matlab:
showRandWalk.m



Possible
Solution

plantUML.com



Lab 3





TurtleBot3 30 Follow Demo



ROBOTIS OpenSourceTeam
13.800 Abonnenten

Abonnieren

👍 41



➦ Teilen

🔖 Speichern

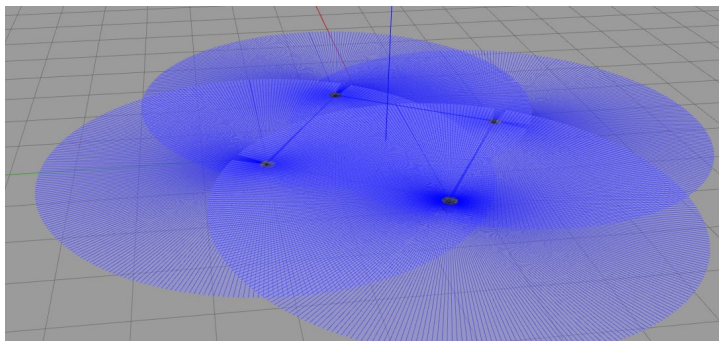


<https://www.youtube.com/watch?v=w9YTxZVY6yQ>

Launch four bots in an empty Gazebo world ...

Download from Moodle and launch

```
~/ros2_ws$ ros2 launch ./my_multi_robot.launch.py
```



```
ros-admin@ros-setup:~/ros2_ws$ ros2 node list
/TB3_1/robot_state_publisher
/TB3_1/turtlebot3_diff_drive
/TB3_1/turtlebot3_imu
/TB3_1/turtlebot3_joint_state
/TB3_1/turtlebot3_laserscan
/TB3_2/robot_state_publisher
/TB3_2/turtlebot3_diff_drive
/TB3_2/turtlebot3_imu
/TB3_2/turtlebot3_joint_state
/TB3_2/turtlebot3_laserscan
/TB3_3/robot_state_publisher
/TB3_3/turtlebot3_diff_drive
/TB3_3/turtlebot3_imu
/TB3_3/turtlebot3_joint_state
/TB3_3/turtlebot3_laserscan
/TB3_4/robot_state_publisher
/TB3_4/turtlebot3_diff_drive
/TB3_4/turtlebot3_imu
/TB3_4/turtlebot3_joint_state
/TB3_4/turtlebot3_laserscan
/gazebo
```

*Delete two bots (e.g., bot2 and bot3) ... uff ... Gazebo name is **burger_2**, but ROS name is **TB3_2** ...*

```
~/ros2_ws$ ros2 service call /delete_entity gazebo_msgs/srv/DeleteEntity "{name: 'burger_2'}"
```

```
waiting for service to become available...
requester: making request: gazebo_msgs.srv.DeleteEntity_Request(name='burger_2')

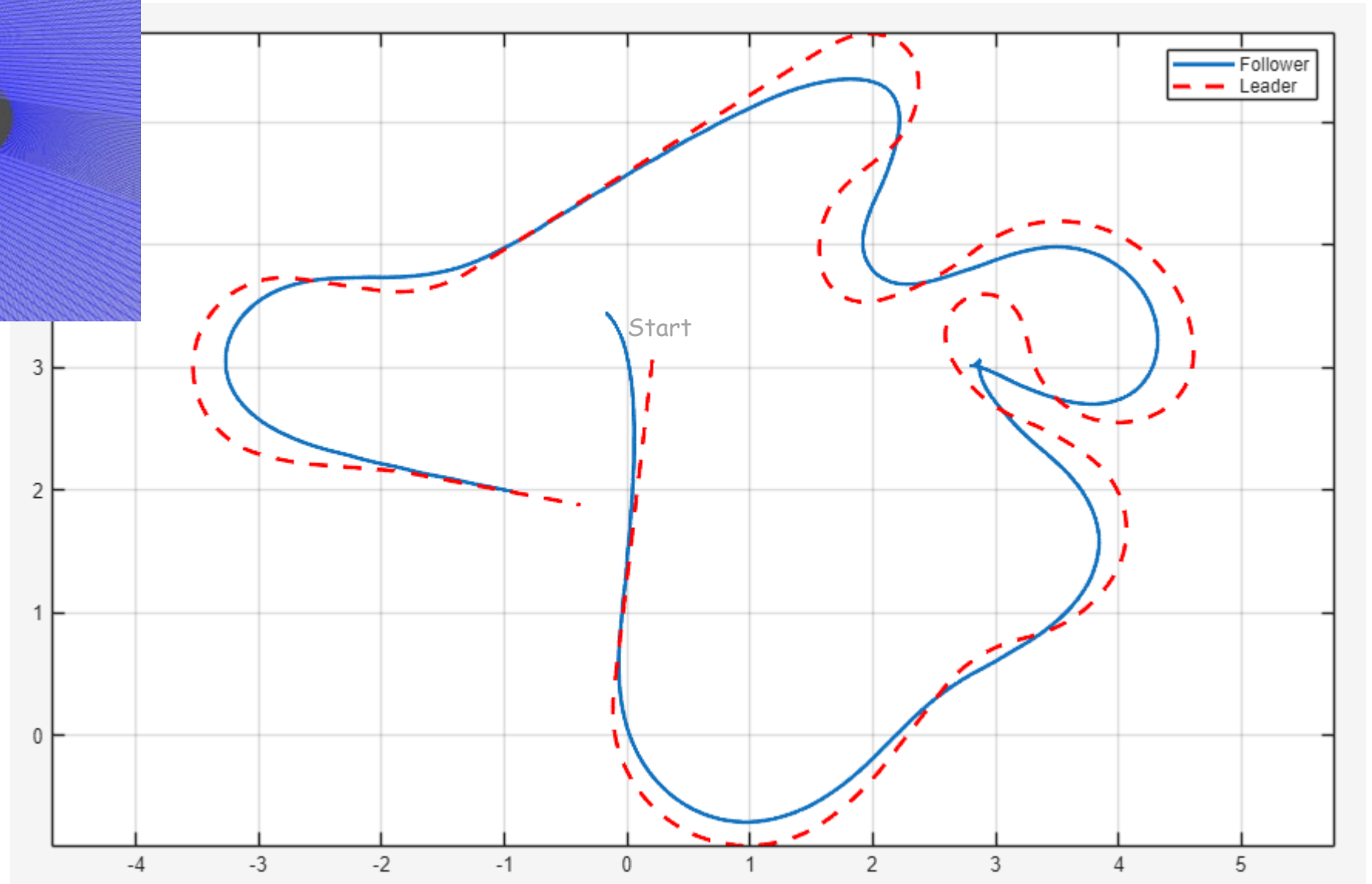
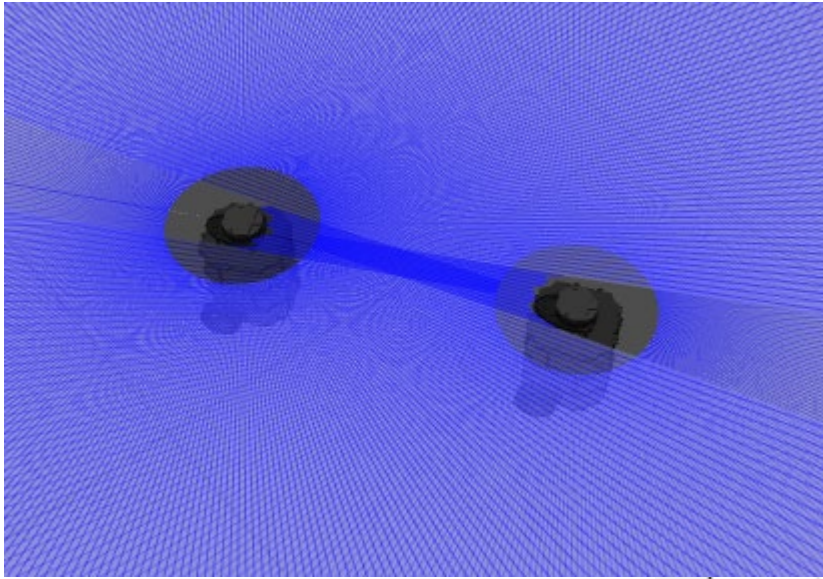
response:
gazebo_msgs.srv.DeleteEntity_Response(success=True, status_message='Successfully deleted entity [burger_2]')
```

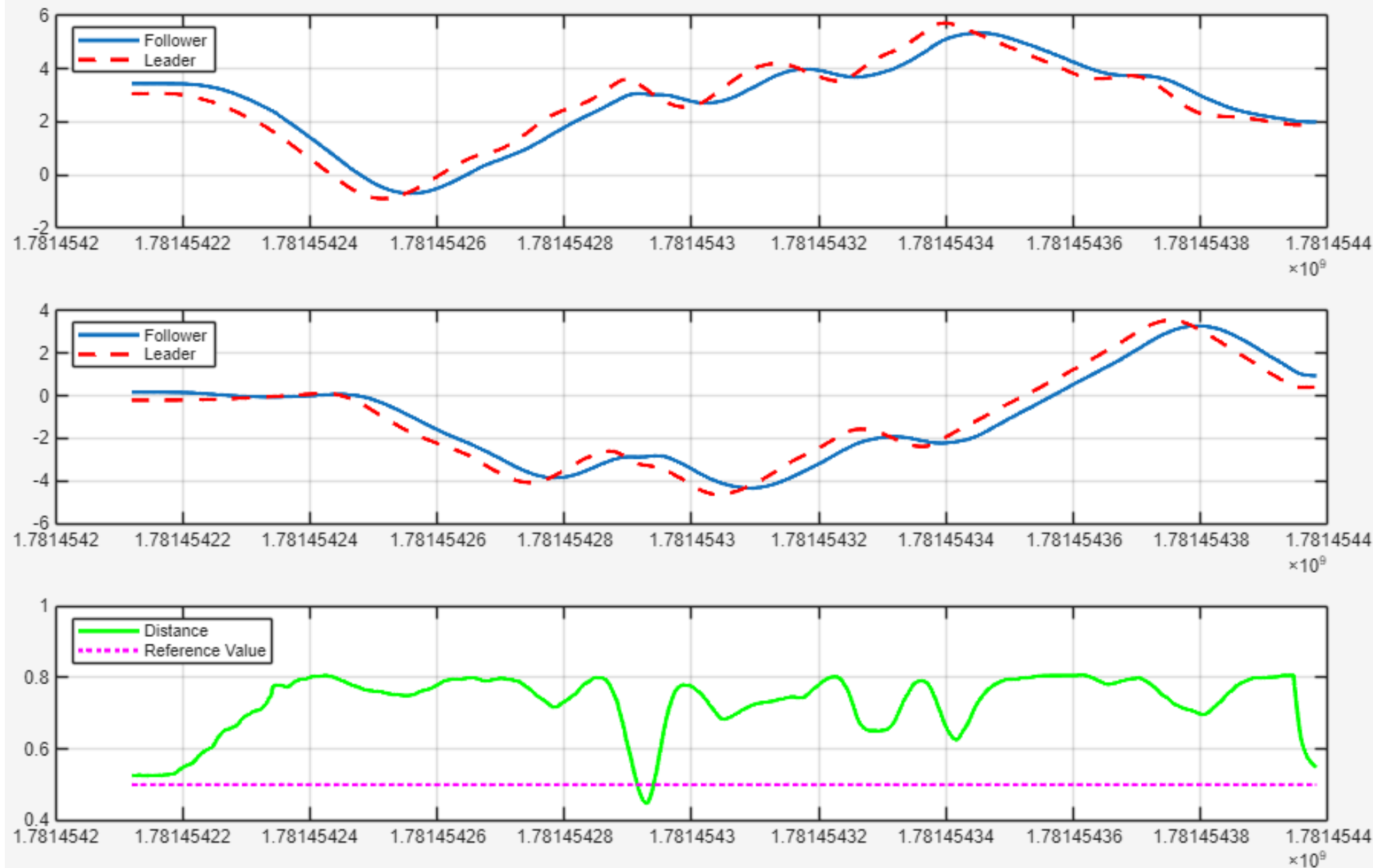
Control one bot (e.g., bot1) via teleop_keyboard

```
~/ros2_ws$ ros2 run turtlebot3_teleop teleop_keyboard --ros-args -r __ns:=/TB3_1
```

Control one bot (e.g., bot 4) via BurgerFollower node

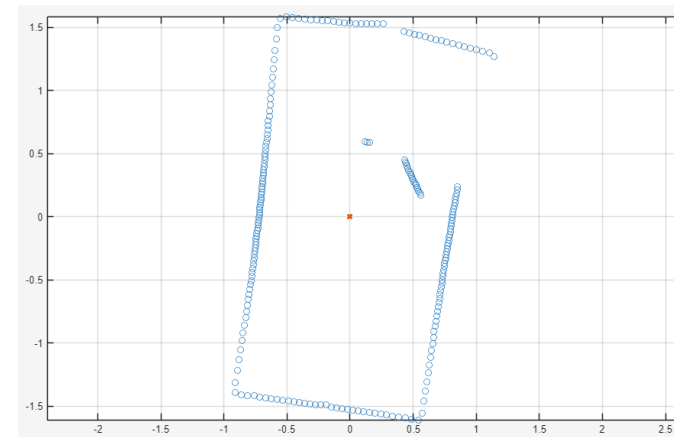
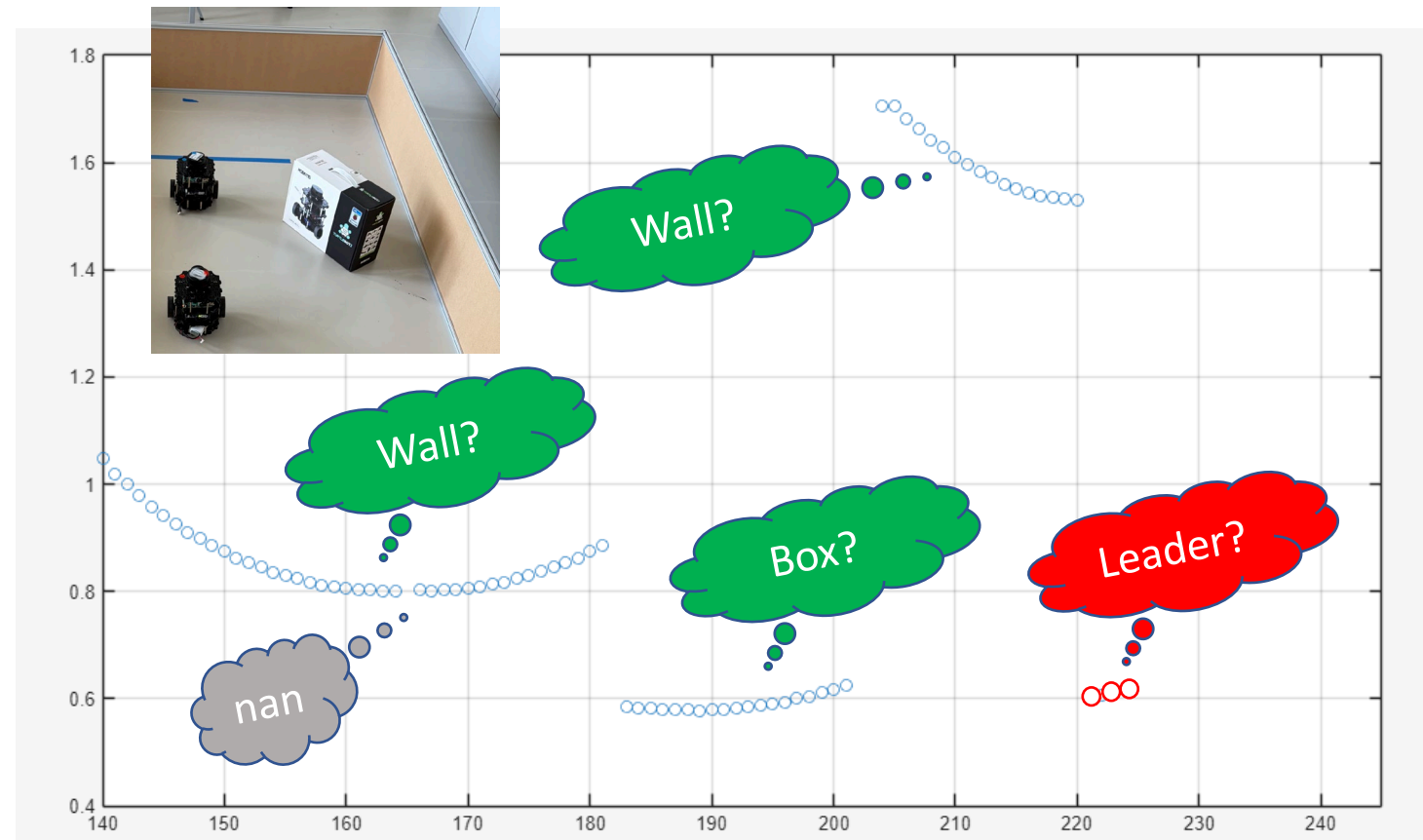
Test Setup
in Gazebo





start with *Gazebo* ... real *Burger* later

- how to detect leader from laser scanner data?
 - clustering points
- what kind of controller?
 - start with P ... I,D later
- what parameter values?
 - reference value == desired distance?
 - how many point in a valid cluster?
 - what to do if leader is lost?
 - collision avoidance necessary?
- what if Laser Scanner is not „enough“ ?
 - Sensor Fusion, Data Fusion
 - (add camera data)
 - Kalman Filter to predict leader's movements
 - use /leader/odom („[breadcrumbs](#)“)



In a robotics leader-follower application, "**breadcrumbs**" (or a breadcrumb trail) refers to a navigation technique where the leader robot regularly drops **virtual coordinates (waypoints)** along its path, which the follower robot then tracks and visits in chronological order.

Just like the fairy tale of Hansel and Gretel, these virtual markers allow the follower to precisely duplicate the leader's exact path rather than trying to follow the leader blindly in a straight line.

Why Breadcrumbs Are Necessary

If a follower robot only tracks the leader's *current* position in real-time, it will cut corners.

- **Without Breadcrumbs:** If the leader turns 90 degrees around a sharp wall corner, a direct line-of-sight follower will try to drive diagonally toward the leader, crashing straight into the wall.
- **With Breadcrumbs:** The follower drives to the exact spot where the leader turned *before* making the turn itself, successfully avoiding the obstacle. 



How It Works in Code and Hardware

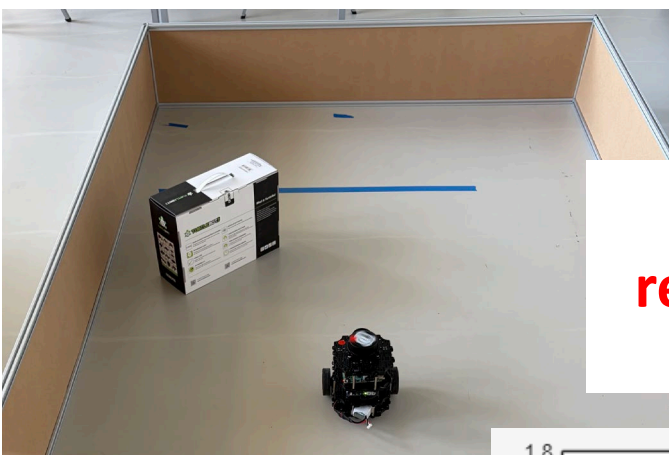
1. **Generation:** As the leader drives, it drops a breadcrumb based on a trigger. This is usually either **distance-based** (e.g., every 20 cm) or **time-based** (e.g., every 0.5 seconds).
2. **Data Structure:** The breadcrumbs are stored in a **FIFO queue** (First-In, First-Out) containing `(x, y)` coordinates, and sometimes orientation (theta).
3. **Consumption:** The follower targets the oldest breadcrumb in the queue. Once it gets within a small acceptance radius (e.g., 5 cm), that breadcrumb is deleted, and the follower targets the next one.
4. **Distance Management:** To prevent the follower from crashing into the leader, the follower stops if the queue runs low on breadcrumbs, maintaining a safe gap.

Methods for Dropping Breadcrumbs

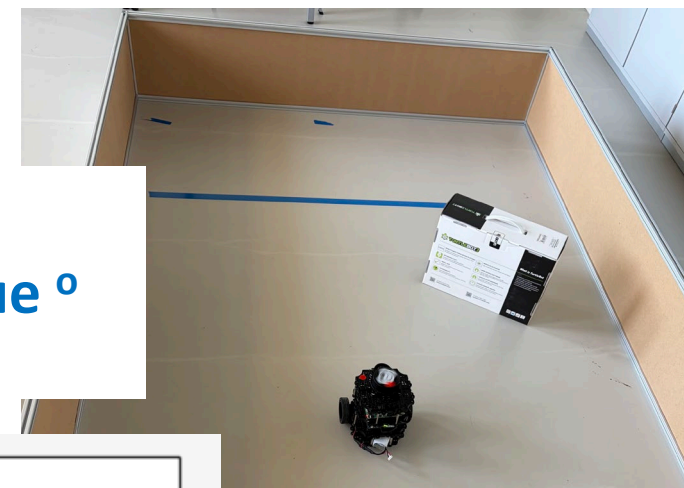
- **Odometry-Based (Virtual):** The leader broadcasts its own calculated `(x,y)` map coordinates via a wireless network (like ROS/ROS2 topics). This is how sensors like your **LDS-02 laser scanner** and wheel encoders help map the path.
- **Physical/Visual:** In environments without GPS or a shared map, the leader can drop physical markers (like AR/AprilTag tags) or use visual odometry to leave a digital trail in a camera's vision field.

Breadcrumbs can result from measurements done by the follower or done by the leader

Lab 3 :: In which direction does the laser scanner rotate?

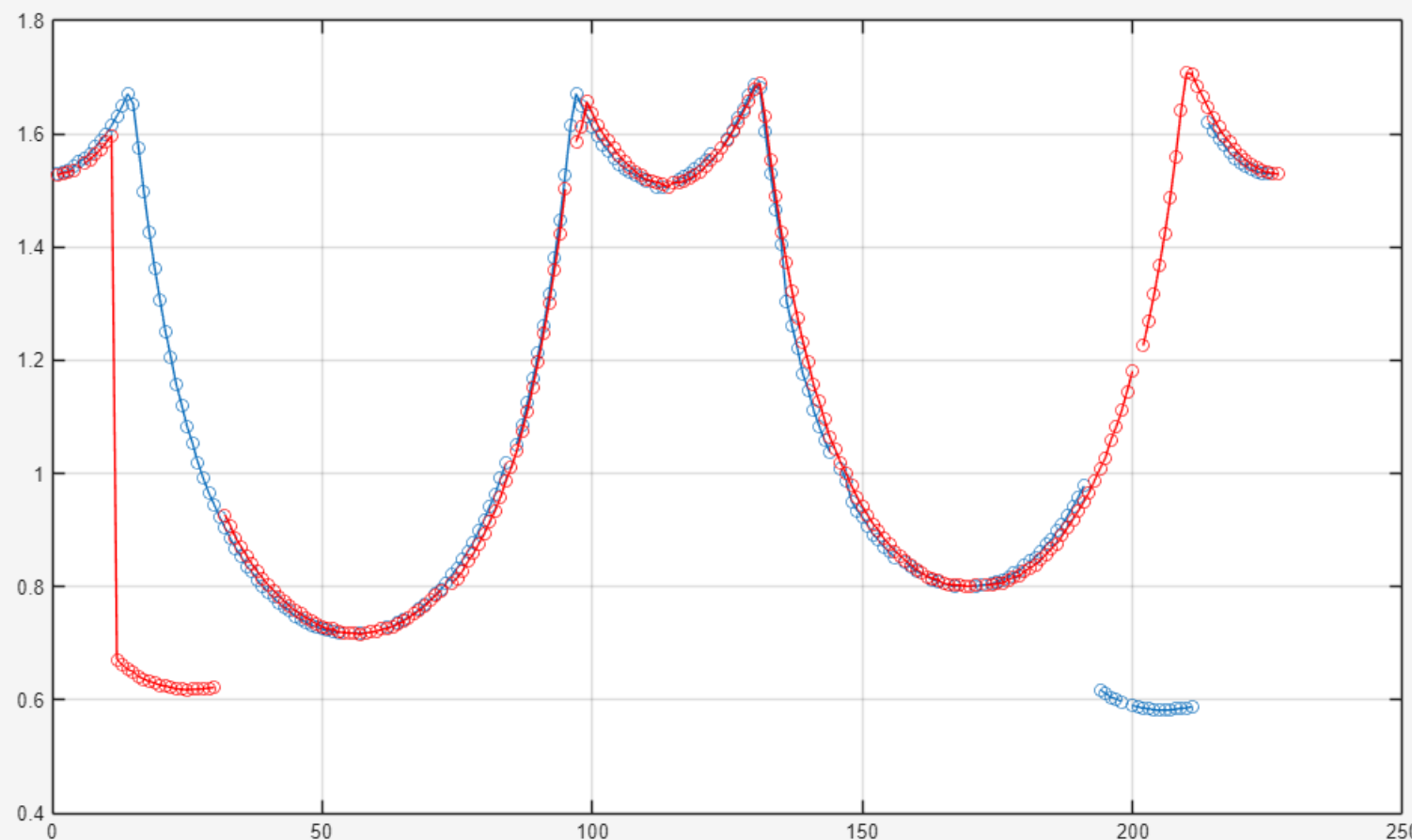


red °



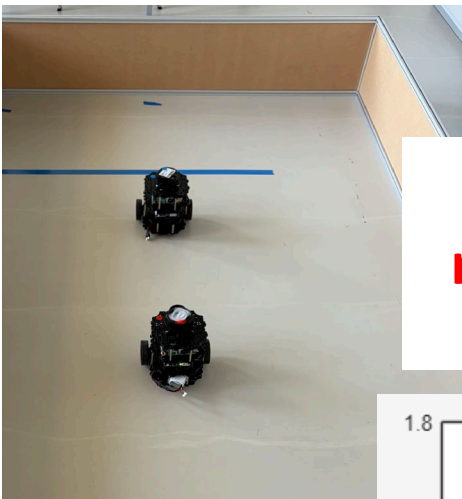
blue °

ranges



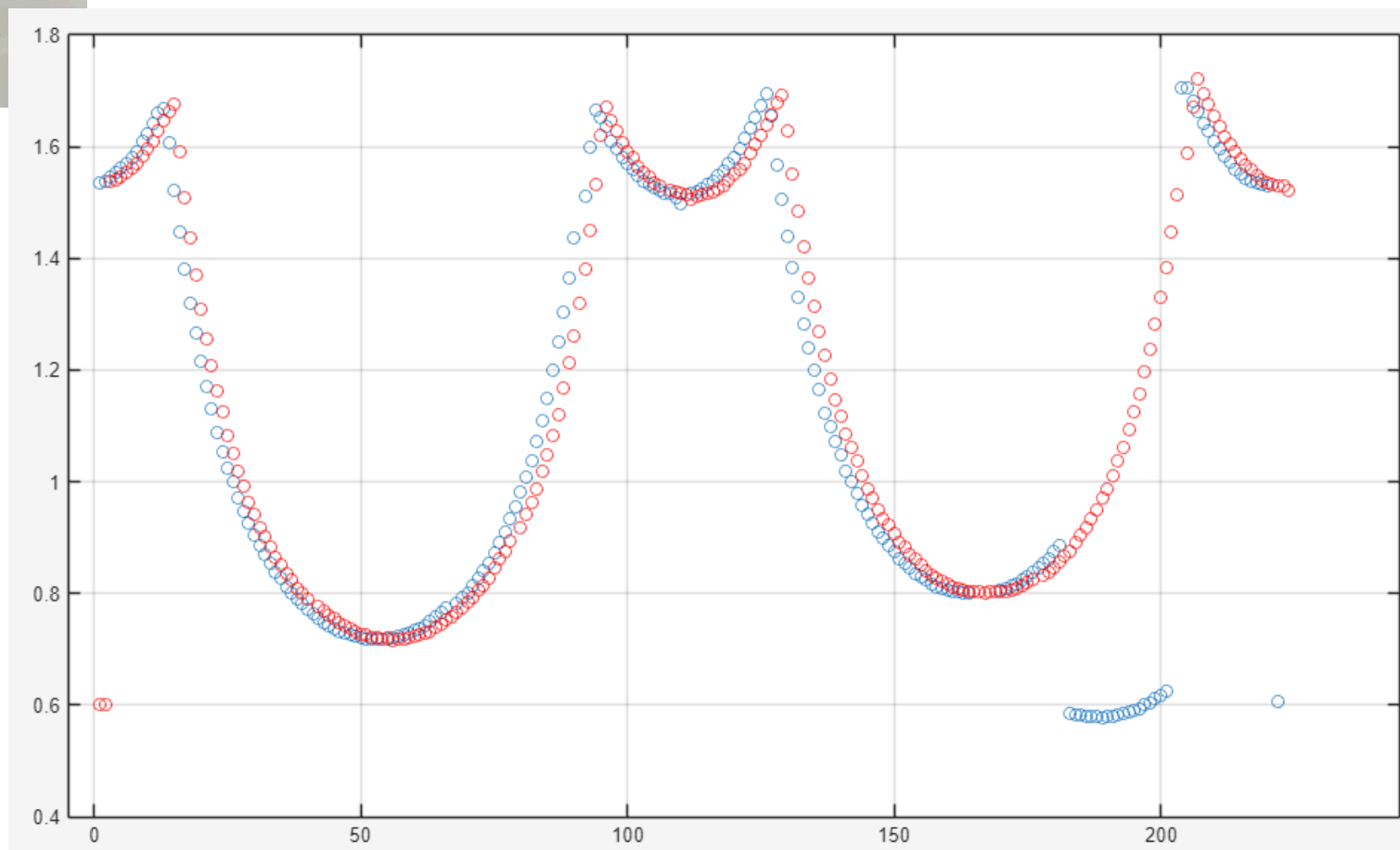
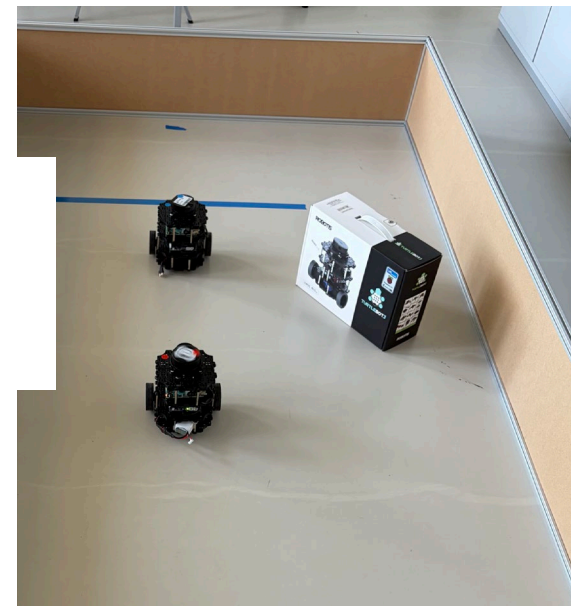
index

Lab 3 :: How many laser beams of the follower catch the leader bot?



red °

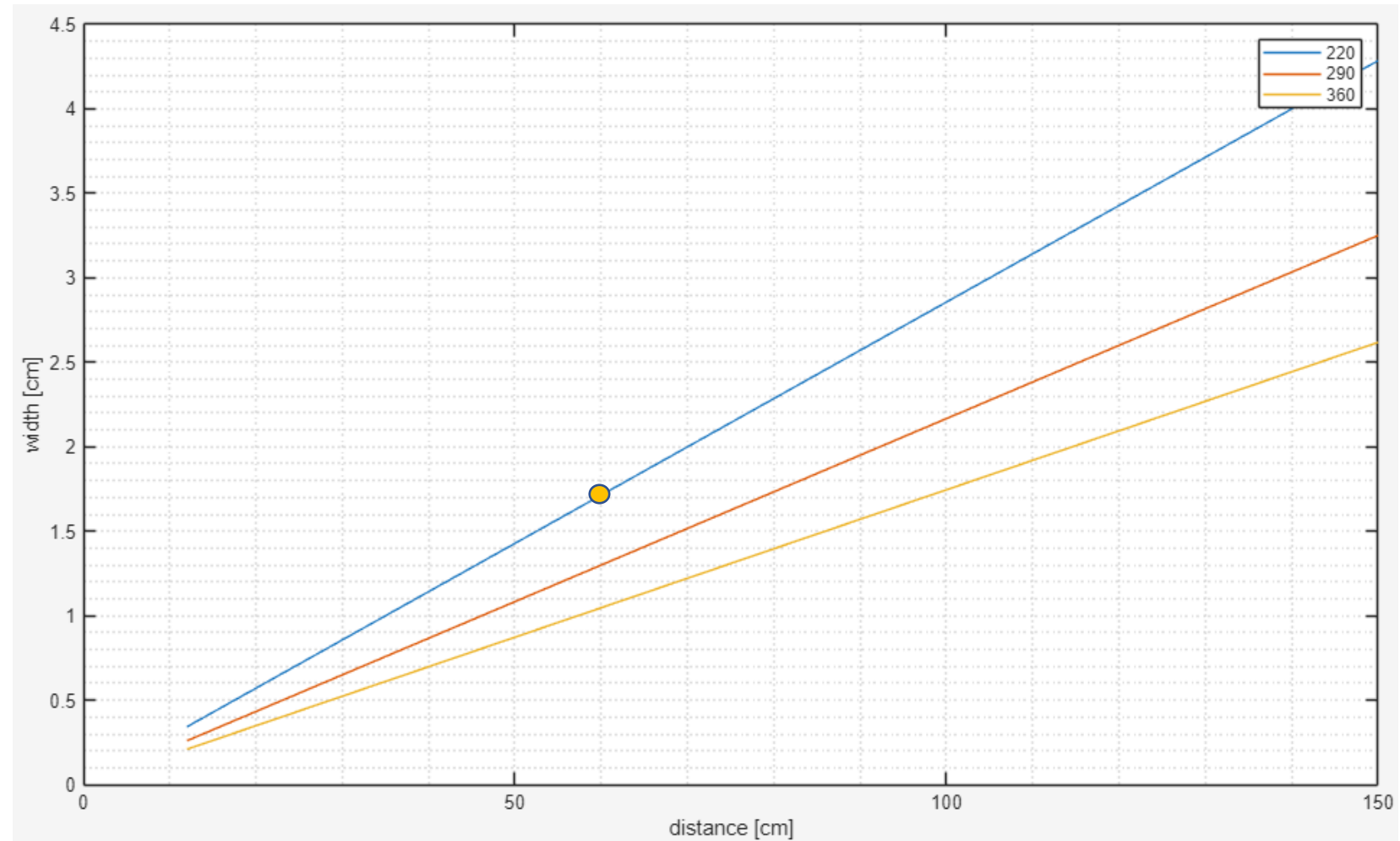
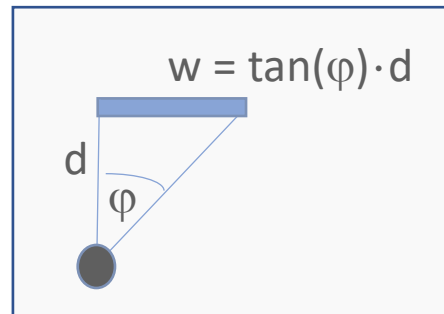
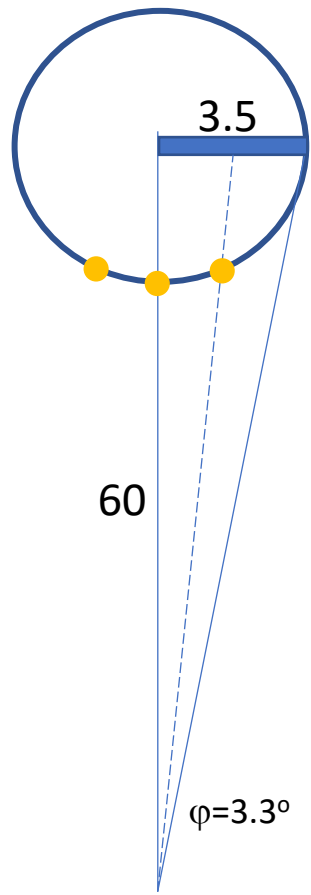
blue °



Lab 3 :: How many laser beams can catch the leader?

According to the official [ROBOTIS TurtleBot3 documentation](#), the LDS-02 360° Laser Distance Sensor has the following physical dimensions and weight:

- Width: 70 mm
- Depth: 90 mm
- Height: 42 mm



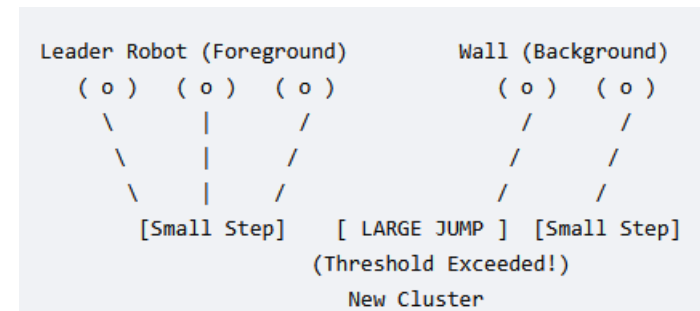
In a ROS2 Humble Python node for a TurtleBot3 Burger, the **Jump Distance Algorithm** (often abbreviated as **JDA**) is a fast, geometric segmentation technique used to group raw LIDAR point clouds (from your **LDS-02 scanner**) into distinct physical objects like walls, legs, or a leader robot.

Because a LIDAR outputs data sequentially sorted by angle, the algorithm evaluates the distance change (the "jump") between consecutive laser beams to find the boundaries of objects. 

How the Algorithm Works geometrically

The LDS-02 scanner on the TurtleBot3 Burger sweeps 360° and returns an array of distances. The algorithm loops through these sequential points and measures the spatial distance between Point P_i and Point P_{i+1} .

1. **The Threshold (δ):** You define a maximum allowed "jump distance" threshold (typically 10 to 30 cm in robotics).
2. **Sequential Check:** If the Euclidean distance between two adjacent points is **less** than the threshold, they belong to the **same object**.
3. **The "Jump":** If the distance is **greater** than the threshold, a "jump" has occurred. This means the laser beam slipped past the edge of a foreground object (like the leader robot) and hit something in the background (like a distant wall). The algorithm cuts the current cluster and starts a brand-new one. 



read
me
later

read
me
later

How to use JDA for Leader-Follower Tracking

Once the JDA has segmented the 360° environment into distinct clusters, your Python node filters through them to isolate the TurtleBot3 Leader:

- **Size Filtering:** A TurtleBot3 Burger chassis is roughly 14 cm wide. You can calculate the bounding width of each cluster and discard anything too small (like table legs) or too wide (like walls).
- **Centroid Tracking:** Calculate the center of mass (mean X and Y) of the surviving cluster. This coordinate becomes your **Leader Position**.
- **Breadcrumb Drop:** Once you have the Leader's (x, y) coordinate relative to the follower, you can save it into your **breadcrumb queue** so your follower can safely navigate to that exact spot.

Why use JDA instead of DBSCAN or K-Means?

While DBSCAN performs a similar neighborhood search, **JDA runs significantly faster** (Complexity $O(N)$ vs DBSCAN's $O(N \log N)$). JDA takes advantage of the fact that the LIDAR data arrives pre-sorted by angle. It only needs a single linear loop through the array to segment the entire room, making it ideal for the low-compute Raspberry Pi on the TurtleBot3 Burger.

Challenges:

- distinguish leader from walls etc.
- handling wrap-around condition
- constant safety distance